

GENERATIVE MODELS

Jiseok Jeong

Department of Electrical Engineering,
Pohang University of Science and Technology

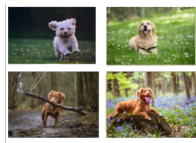
April 27, 2026

Part I

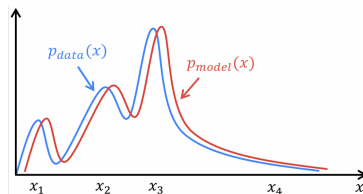
OVERVIEW

WHAT IS A GENERATIVE MODEL?

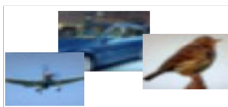
1. Assume that the training data $\{\mathbf{x}_i\}_{i=1}^n$ is generated from the probability distribution $p_{\text{data}}(\mathbf{x}) \Rightarrow \mathbf{x} \sim p_{\text{data}}(\mathbf{x})$
2. Train a parameterized model $p_{\theta}(\mathbf{x}) \Rightarrow p_{\theta}(\mathbf{x}) \approx p_{\text{data}}(\mathbf{x})$
3. Generate new data from the learned $p_{\theta}(\mathbf{x}) \Rightarrow \mathbf{x} \sim p_{\theta}(\mathbf{x})$



$\mathbf{x}_i \sim p_{\text{data}}$
 $i = 1, 2, \dots, n$



Step 1: density estimation



$p_{\theta}(\mathbf{x})$

Step 2: sampling



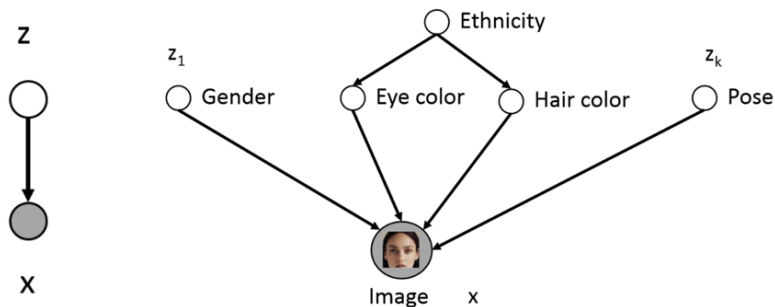
LATENT VARIABLE MODEL

Assume the data has some underlying structure captured by a latent variable $\mathbf{z}$ drawn from a simple distribution. A decoder network then maps $\mathbf{z}$ to $\mathbf{x}$:

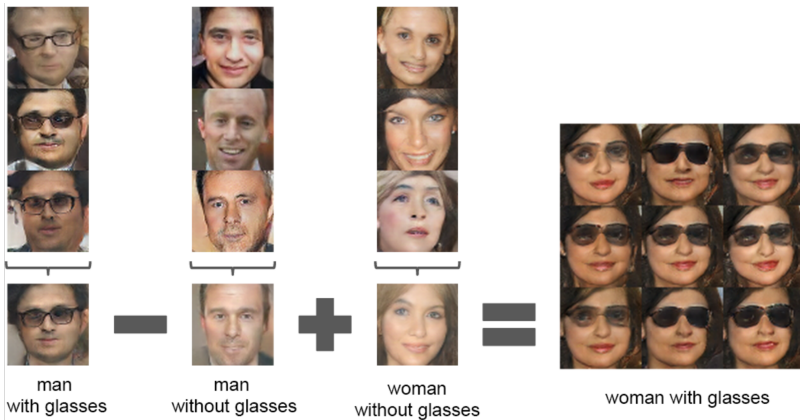
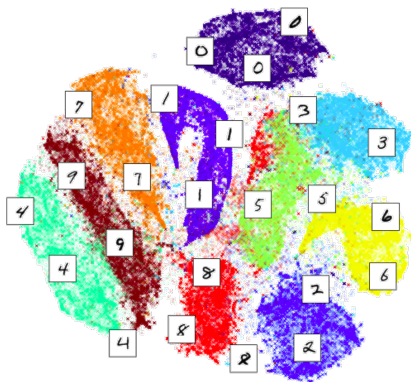
$$\mathbf{z} \sim p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad \mathbf{x}|\mathbf{z} \sim p_{\theta}(\mathbf{x}|\mathbf{z}) = \mathcal{N}(D_{\theta}(\mathbf{z}), \sigma^2 \mathbf{I})$$

$$p_{\theta}(\mathbf{x}) = \int p_{\theta}(\mathbf{x}|\mathbf{z}) p(\mathbf{z}) d\mathbf{z}$$

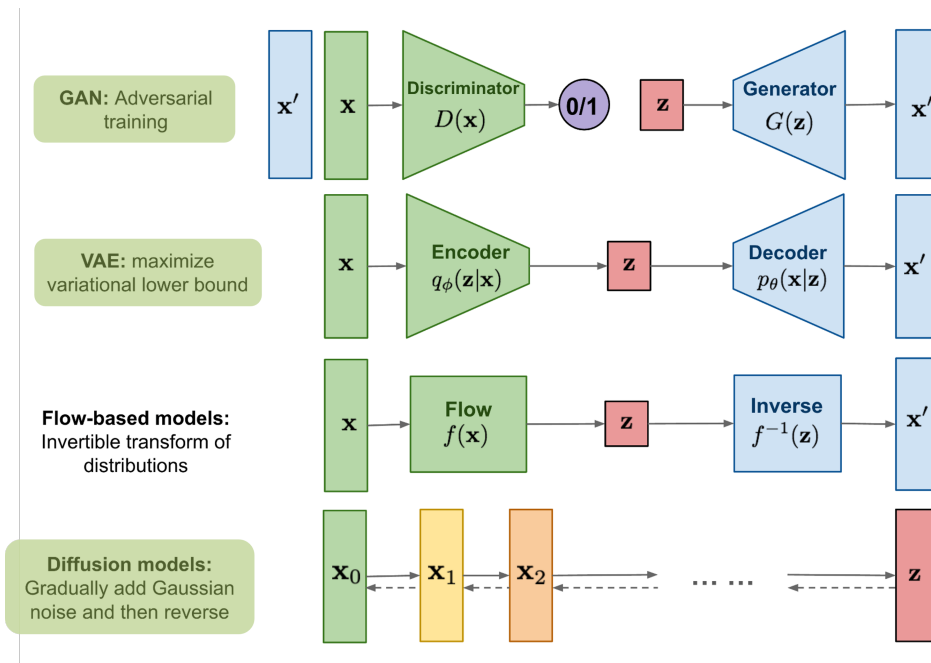
Even though $p(\mathbf{z})$ and $p_{\theta}(\mathbf{x}|\mathbf{z})$ are each simple Gaussians, the mixture over all possible $\mathbf{z}$ can represent complex, multimodal distributions.



LATENT VARIABLE MODEL



LATENT VARIABLE MODEL

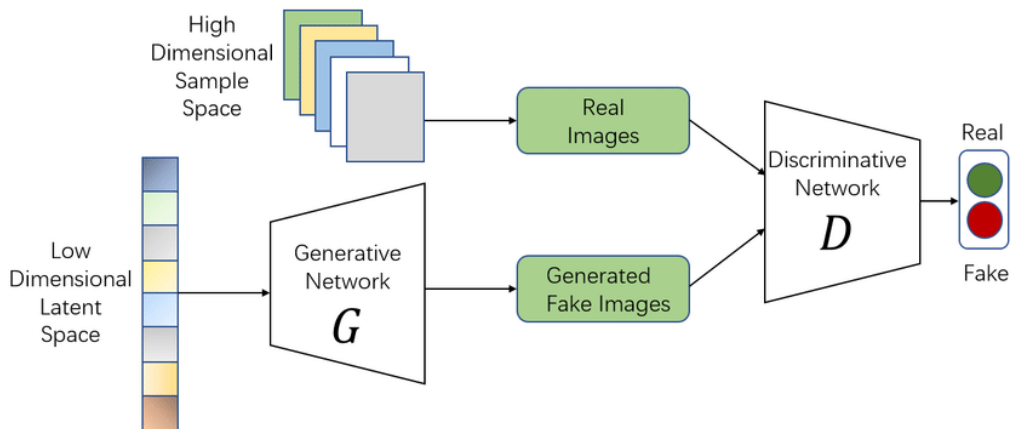


Part II

GENERATIVE ADVERSARIAL NETWORK (GAN)

GENERATIVE ADVERSARIAL NETWORK

- ▶ Generator (or decoder) G takes a latent sampled from a unit Gaussian as input and generates a synthetic image.
- ▶ Discriminator D takes an image as input and classifies it as either real or fake (generated).



OBJECTIVE FUNCTION

- ▶ A min-max optimization problem, which is known as very difficult to solve

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Sample x from real data distribution Sample latent code z from Gaussian distribution

$$\min_G \max_D V(D, G) = E_{x \sim p_{\text{data}}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

D should maximize $V(D, G)$ Maximum when $D(x) = 1$ Maximum when $D(G(z)) = 0$

G is independent of this part

$$\min_G \max_D V(D, G) = \cancel{E_{x \sim p_{\text{data}}(x)} [\log D(x)]} + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

G should minimize $V(D, G)$ Minimum when $D(G(z)) = 1$

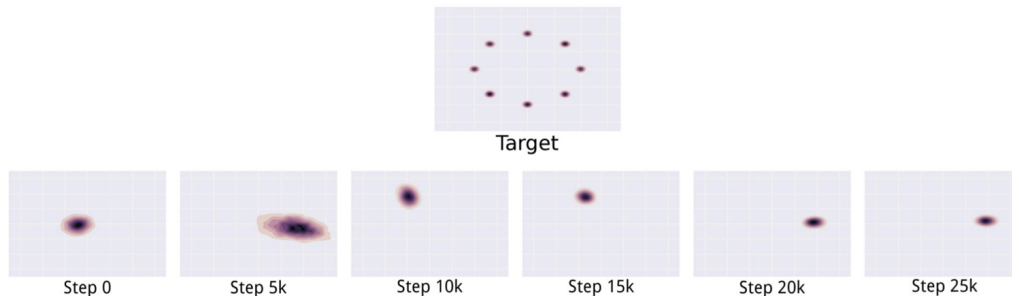
CHALLENGES IN GAN TRAINING

1. Gradient Vanishing

- The discriminator becomes too strong and can easily distinguish between real and fake samples.
- This leads to near-zero gradients, halting the generator's learning process.

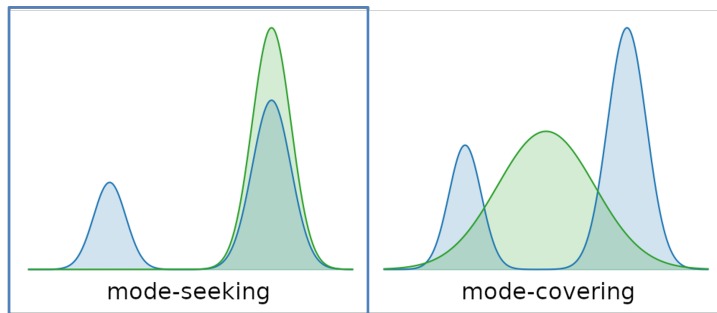
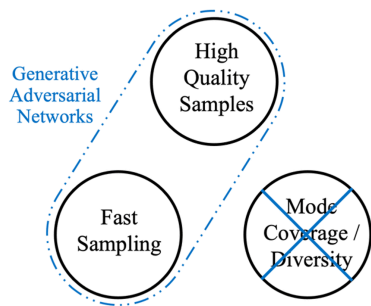
2. Mode Collapse

- The generator tends to collapse to a few modes of the data distribution instead of capturing the full diversity of the true data distribution.



SUMMARY OF GAN

The Generative Learning Trilemma



Part III

VARIATIONAL AUTOENCODER (VAE)

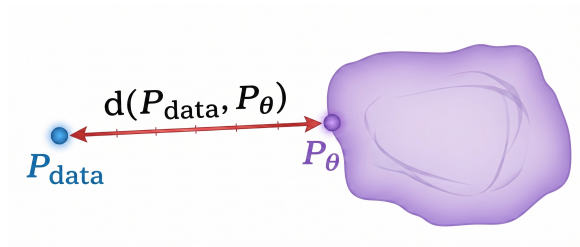
MAXIMUM LIKELIHOOD LEARNING

We want p_θ close to p_{data} . Using KL divergence as the measure:

$$D_{\text{KL}}(p_{\text{data}} \parallel p_\theta) = \underbrace{\mathbb{E}_{x \sim p_{\text{data}}} [\log p_{\text{data}}(x)]}_{\text{const w.r.t. } \theta} - \mathbb{E}_{x \sim p_{\text{data}}} [\log p_\theta(x)]$$

$$\min_{\theta} D_{\text{KL}}(p_{\text{data}} \parallel p_\theta) \iff \max_{\theta} \frac{1}{N} \sum_{i=1}^N \log p_\theta(x_i) \quad (\text{MLE})$$

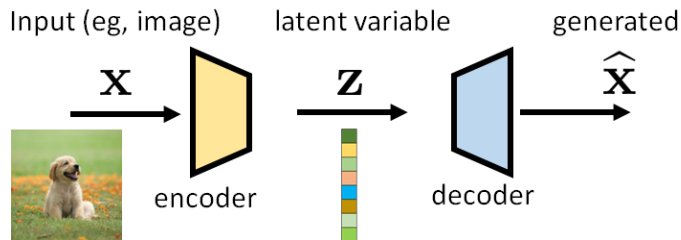
$\implies$ training = maximizing $\log p_\theta(x)$.



VARIATIONAL AUTOENCODER

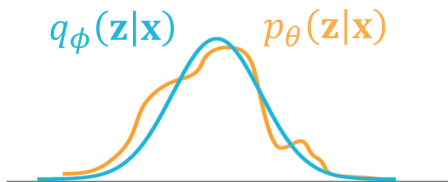
The model consists of two components:

- **Encoder** $q_{\phi}(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}; \mu_{\phi}(\mathbf{x}), \sigma_{\phi}^2(\mathbf{x})\mathbf{I})$: maps input $\mathbf{x}$ to a distribution over the latent space (mean μ and variance σ^2).
- **Decoder** $p_{\theta}(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mathbf{x}; D_{\theta}(\mathbf{z}), \sigma^2\mathbf{I})$: maps a sampled latent vector $\mathbf{z}$ back to the data space, reconstructing $\hat{\mathbf{x}}$.



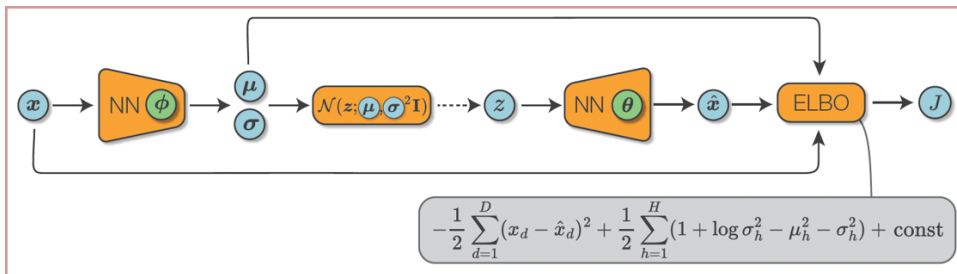
OBJECTIVE FUNCTION: ELBO DERIVATION

$$\begin{aligned}\log p_{\theta}(\mathbf{x}) &= \int_{\mathbf{z}} q_{\phi}(\mathbf{z}|\mathbf{x}) \log p_{\theta}(\mathbf{x}) d\mathbf{z} = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p_{\theta}(\mathbf{x})] \\&= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{p_{\theta}(\mathbf{z}|\mathbf{x})} \right] \\&= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{p_{\theta}(\mathbf{z}|\mathbf{x})} \cdot \frac{q_{\phi}(\mathbf{z}|\mathbf{x})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right] \\&= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right] + \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{q_{\phi}(\mathbf{z}|\mathbf{x})}{p_{\theta}(\mathbf{z}|\mathbf{x})} \right] \\&= \underbrace{\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right]}_{\text{ELBO}} + D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) || p_{\theta}(\mathbf{z}|\mathbf{x}))\end{aligned}$$



OBJECTIVE FUNCTION: ELBO DECOMPOSITION

$$\begin{aligned}
 \text{ELBO} &= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} \right] = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_\theta(\mathbf{x}|\mathbf{z}) p_\theta(\mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} \right] \\
 &= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \parallel p_\theta(\mathbf{z})) \\
 &\approx \underbrace{-\frac{1}{2} \sum_{d=1}^D (x_d - \hat{x}_d)^2}_{\text{reconstruction term}} + \underbrace{\frac{1}{2} \sum_{h=1}^H (1 + \log \sigma_h^2 - \mu_h^2 - \sigma_h^2)}_{\text{prior matching term}} + \text{const}
 \end{aligned}$$

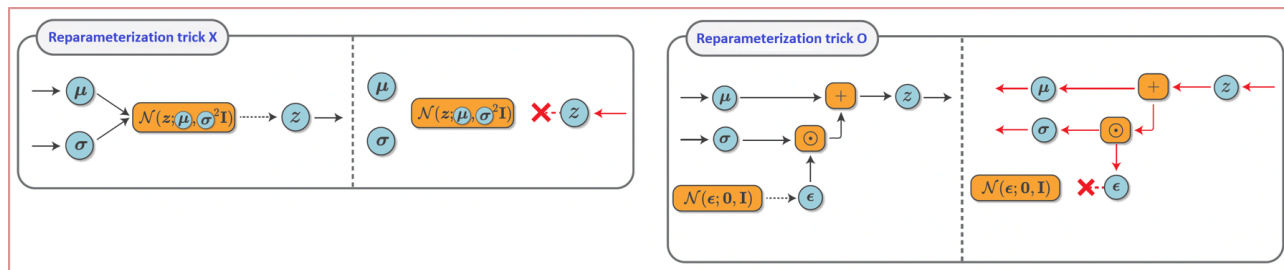


REPARAMETERIZATION TRICK

Stochastic sampling procedure → Non-differentiable

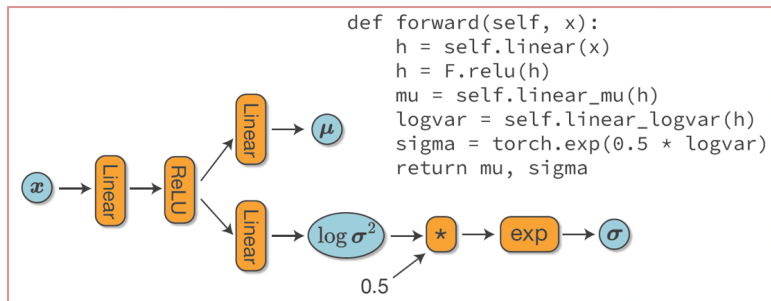
$$\mathbf{z} \sim \mathcal{N}(\mu, \sigma^2 \mathbf{I}) \Rightarrow \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad \mathbf{z} = \mu + \sigma \odot \epsilon = \begin{bmatrix} \mu_1 \\ \mu_2 \\ \vdots \\ \mu_H \end{bmatrix} + \begin{bmatrix} \sigma_1 \\ \sigma_2 \\ \vdots \\ \sigma_H \end{bmatrix} \odot \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_H \end{bmatrix}$$

$$\mathbb{E}[z_i] = \mu_i + \sigma_i \mathbb{E}[\epsilon_i] = \mu_i, \quad \text{Var}(z_i) = \sigma_i^2 \text{Var}(\epsilon_i) = \sigma_i^2$$

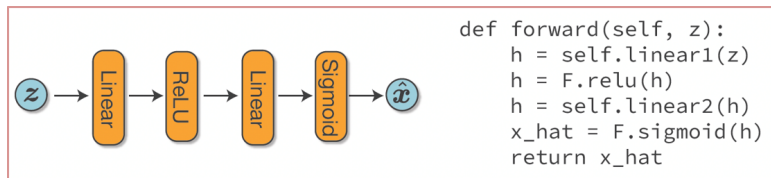


IMPLEMENTATION OF VAE

Encoder

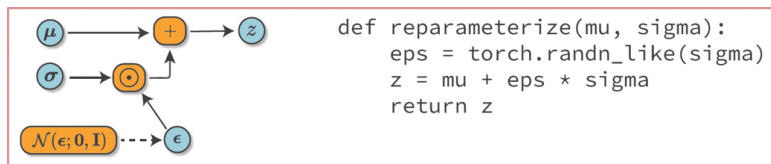


Dcoder

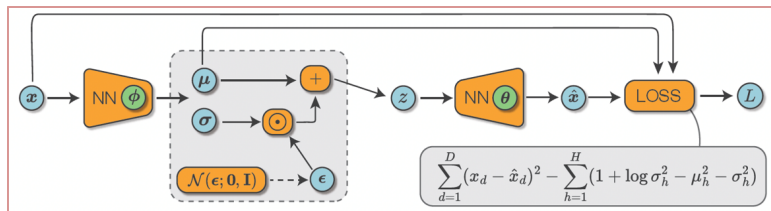


IMPLEMENTATION OF VAE

Reparameterization Trick

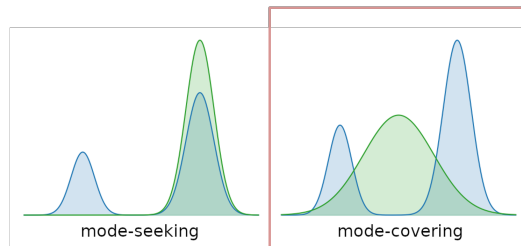
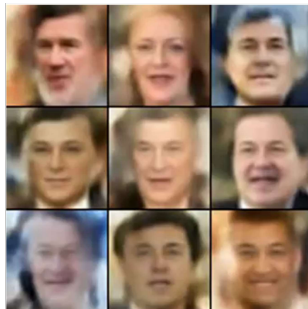
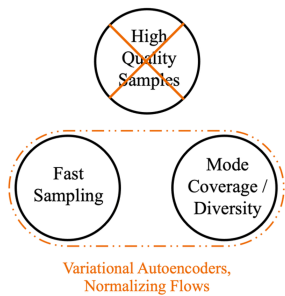


Loss Function



SUMMARY OF VAE

The Generative Learning Trilemma



Part IV

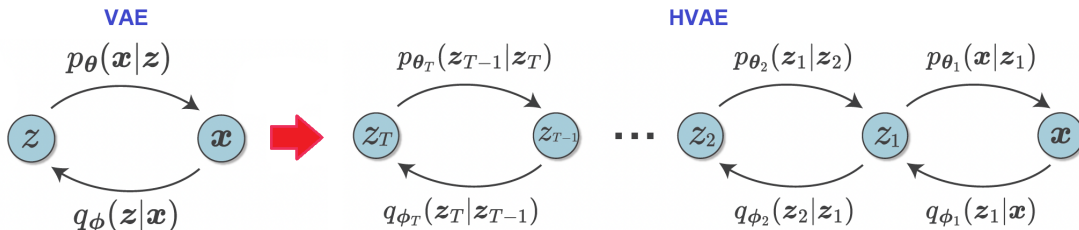
HIERARCHICAL VARIATIONAL AUTOENCODER (HVAE)

EXTENDING VAE TO MULTIPLE LATENT LAYERS

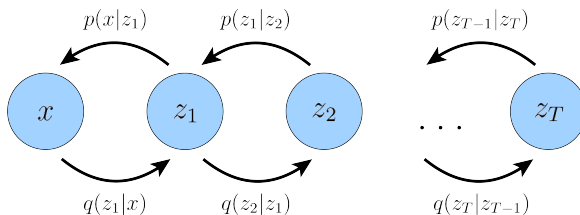
VAE uses a single latent layer $\mathbf{z}$. Two structural limitations:

- ▶ A single $\mathbf{z}$ must simultaneously encode all levels of abstraction (shape, texture, identity, ...).
- ▶ The decoder reconstructs $\mathbf{x}$ from $\mathbf{z}$ in a single step.

A natural extension: stack multiple latent layers $\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_T$, each capturing structure at a different level of abstraction.



HIERARCHICAL VARIATIONAL AUTOENCODER



Markov assumption: each layer depends only on its adjacent layer.

Forward process (encoder):

$$q_{\phi}(\mathbf{x}_{1:T}|\mathbf{x}) = q_{\phi_1}(\mathbf{x}_1|\mathbf{x}) \prod_{t=2}^T q_{\phi_t}(\mathbf{x}_t|\mathbf{x}_{t-1})$$

Reverse process (decoder):

$$p_{\theta}(\mathbf{x}, \mathbf{x}_{1:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta_t}(\mathbf{x}_{t-1}|\mathbf{x}_t)$$

- ▶ $p(\mathbf{x}_T) = \mathcal{N}(\mathbf{0}, \mathbf{I})$: simple prior
- ▶ Each encoder q_{ϕ_t} and decoder p_{θ_t} is a learnable neural network

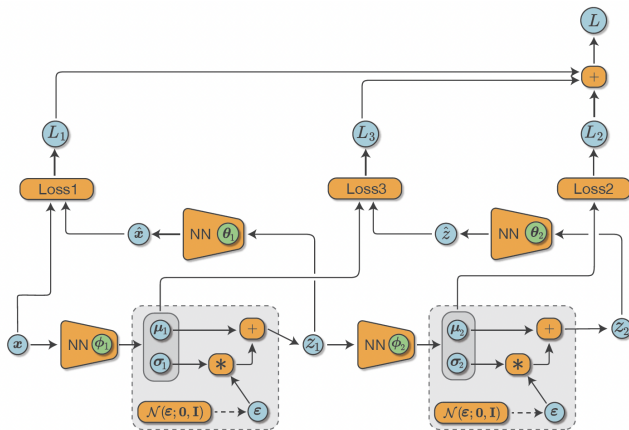
OBJECTIVE FUNCTION: ELBO

The same variational strategy as VAE applies.

$$\begin{aligned}\log p_\theta(\mathbf{x}_0) &= \log \int p_\theta(\mathbf{x}_{0:T}) d\mathbf{x}_{1:T} \\&= \log \int \frac{p_\theta(\mathbf{x}_{0:T})}{q_\phi(\mathbf{x}_{1:T}|\mathbf{x}_0)} \cdot q_\phi(\mathbf{x}_{1:T}|\mathbf{x}_0) d\mathbf{x}_{1:T} \\&= \log \mathbb{E}_{q_\phi} \left[\frac{p_\theta(\mathbf{x}_{0:T})}{q_\phi(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] \\&\geq \underbrace{\mathbb{E}_{q_\phi} \left[\log \frac{p_\theta(\mathbf{x}_{0:T})}{q_\phi(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right]}_{\text{ELBO}} \\&= \underbrace{\mathbb{E}_{q_\phi} [\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)]}_{\mathcal{L}_0: \text{reconstruction}} - \underbrace{D_{\text{KL}}(q_\phi(\mathbf{x}_T|\mathbf{x}_{T-1}) \parallel p(\mathbf{x}_T))}_{\mathcal{L}_T: \text{prior matching}} - \sum_{t=1}^{T-1} \underbrace{\mathbb{E}_{q_\phi} D_{\text{KL}}(q_\phi(\mathbf{x}_t|\mathbf{x}_{t-1}) \parallel p_\theta(\mathbf{x}_t|\mathbf{x}_{t+1}))}_{\mathcal{L}_{t-1}: \text{consistency}}\end{aligned}$$

TWO-LEVEL HVAE

$$\begin{aligned}
 \text{ELBO} &= \mathbb{E}_{q_{\phi_1}} [\log p_{\theta_1}(\mathbf{x}|\mathbf{x}_1)] - \mathbb{E}_{q_{\phi_1}} [D_{\text{KL}}(q_{\phi_2}(\mathbf{x}_2|\mathbf{x}_1) \| p(\mathbf{x}_2))] - \mathbb{E}_{q_{\phi_2}} [D_{\text{KL}}(q_{\phi_1}(\mathbf{x}_1|\mathbf{x}) \| p_{\theta_2}(\mathbf{x}_1|\mathbf{x}_2))] \\
 &\approx -\frac{1}{2} \sum_{d=1}^D (\mathbf{x}_d - \hat{x}_d)^2 + \frac{1}{2} \sum_{h=1}^H (1 + \log \sigma_{2,h}^2 - \mu_{2,h}^2 - \sigma_{2,h}^2) + \frac{1}{2} \sum_{h=1}^H (1 + \log \sigma_{1,h}^2 - (\mu_{1,h} - \hat{z}_{1,h})^2 - \sigma_{1,h}^2)
 \end{aligned}$$



Part V

DENOISING DIFFUSION PROBABILISTIC MODELS (DDPM)

HVAE $\rightarrow$ DDPM

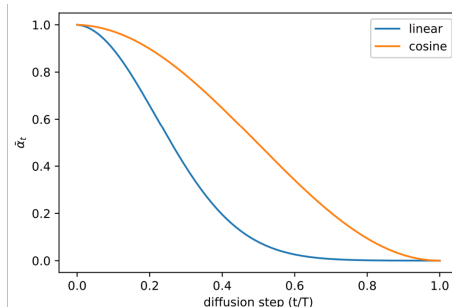
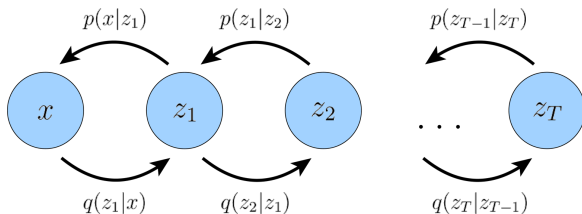
DDPM is a Markovian HVAE with three key restrictions:

1. **Data dimension = Latent dimension** $\Rightarrow \dim(\mathbf{x}) = \dim(\mathbf{x}_t)$ for all t
2. **The encoder is fixed — not learned**
 - A pre-defined linear Gaussian transition

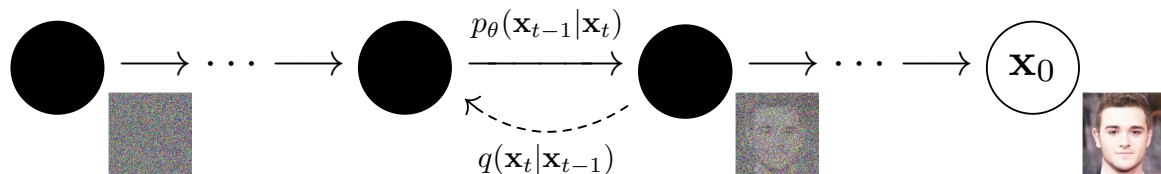
$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t} \mathbf{x}_{t-1}, (1 - \alpha_t) \mathbf{I})$$

3. **The final latent follows a standard Gaussian**

$$\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$



FORWARD & REVERSE PROCESS



- ▶ **Forward Process:** The process of gradually adding Gaussian noise to $\mathbf{x}_0$ to produce $\mathbf{x}_T$, which is fully noised.
- ▶ **Reverse Process:** The process of generating an image by gradually removing noise from $\mathbf{x}_T$.
 - When each forward step is sufficiently small, the reverse step $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ is also approximately Gaussian.

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I})$$

FORWARD PROCESS

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t} \mathbf{x}_{t-1}, (1 - \alpha_t) \mathbf{I})$$

$$\begin{aligned} \mathbf{x}_t &= \sqrt{\alpha_t} \mathbf{x}_{t-1} + \sqrt{1 - \alpha_t} \epsilon_{t-1} \\ &= \sqrt{\alpha_t} (\sqrt{\alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{1 - \alpha_{t-1}} \epsilon_{t-2}) + \sqrt{1 - \alpha_t} \epsilon_{t-1} \\ &= \sqrt{\alpha_t \alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{\alpha_t - \alpha_t \alpha_{t-1}} \epsilon_{t-2} + \sqrt{1 - \alpha_t} \epsilon_{t-1} \\ &= \sqrt{\alpha_t \alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{1 - \alpha_t \alpha_{t-1}} \epsilon_{t-2} \\ &\vdots \\ &= \sqrt{\prod_{i=1}^t \alpha_i} \mathbf{x}_0 + \sqrt{1 - \prod_{i=1}^t \alpha_i} \epsilon_0 \end{aligned}$$

Define $\bar{\alpha}_t := \prod_{i=1}^t \alpha_i$.

$$\begin{aligned} q(\mathbf{x}_t | \mathbf{x}_0) &= \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}) \\ \mathbf{x}_t &= \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \end{aligned}$$

DDPM ELBO

Applying the Markovian HVAE ELBO to DDPM:

$$\text{ELBO} = \underbrace{\mathbb{E}_q[\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)]}_{\mathcal{L}_0: \text{reconstruction}} - \underbrace{D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \parallel p(\mathbf{x}_T))}_{\mathcal{L}_T: \approx 0} - \sum_{t=2}^T \underbrace{\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{\mathcal{L}_{t-1}: \text{denoising matching}}$$

- ▶ Since the forward process is fixed (not learned), the encoder is no longer q_ϕ but simply q .
- ▶ $\mathcal{L}_T \approx 0$: noise schedule designed so that $\bar{\alpha}_T \approx 0$.

FORWARD POSTERIOR $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$

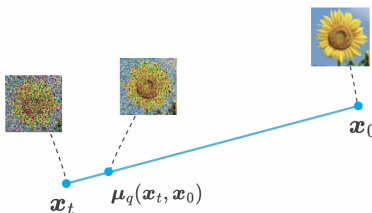
Since $q(\mathbf{x}_t|\mathbf{x}_{t-1})$, $q(\mathbf{x}_{t-1}|\mathbf{x}_0)$, and $q(\mathbf{x}_t|\mathbf{x}_0)$ are all Gaussian, Bayes' rule gives a closed-form Gaussian:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1}) q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)} = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I})$$

$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)}{1 - \bar{\alpha}_t} \mathbf{x}_0, \quad \tilde{\beta}_t = \frac{(1 - \bar{\alpha}_{t-1})(1 - \alpha_t)}{1 - \bar{\alpha}_t}$$

The denoising matching term simplifies to mean matching (both distributions are Gaussian with fixed variance):

$$\mathcal{L}_{t-1} \propto \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} \left[\|\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) - \mu_\theta(\mathbf{x}_t, t)\|^2 \right]$$



THREE PREDICTION TARGETS

$$\tilde{\mu}_t = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)}{1 - \bar{\alpha}_t} \mathbf{x}_0, \quad \mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

- ▶ Mean predictor: $\mathcal{L} = \mathbb{E}[\|\tilde{\mu}_t - \mu_\theta(\mathbf{x}_t, t)\|^2]$
- ▶ Original-sample predictor: $\mathcal{L} = \mathbb{E}[\|\mathbf{x}_0 - \hat{\mathbf{x}}_\theta(\mathbf{x}_t, t)\|^2]$
- ▶ Noise predictor: $\mathcal{L} = \mathbb{E}[\|\epsilon - \hat{\epsilon}_\theta(\mathbf{x}_t, t)\|^2]$

SUMMARY OF DDPM

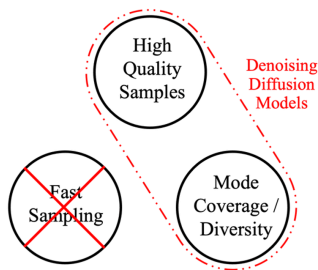
Algorithm 1 Training

- 1: **repeat**
 - 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
 - 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
 - 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 5: Take gradient descent step on
$$\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t)\|^2$$
 - 6: **until** converged
-

Algorithm 2 Sampling

- 1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 2: **for** $t = T, \dots, 1$ **do**
 - 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
 - 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
 - 5: **end for**
 - 6: **return** $\mathbf{x}_0$
-

The Generative Learning Trilemma



Thank you